

INFORME TÉCNICO

METAScore HPC

**OPTIMIZACIÓN PARALELA DEL SISTEMA DE SCORING METAGENÓMICO PARA
CLASIFICACIÓN BINARIA**

**PRESENTADO POR:
TANIA BOTINA 22736**

**UNIVERSIDAD DE CALDAS
INGENIERÍA DE SISTEMAS
PROGRAMACIÓN CONCURRENTE Y DISTRIBUIDA
JUNIO 2026**

MetaScore HPC

Optimización Paralela del Sistema de Scoring Metagenómico para Clasificación Binaria

Asignatura: Programación Concurrente y Distribuida 2026-I

Área: Bioinformática Computacional

Hardware: AMD Ryzen 5 5500U (6 núcleos / 12 lógicos) · NVIDIA T4 via Google Colab

Stack: Python · C/OpenMP · C/MPI · CUDA

Fecha: Junio 2026

ÍNDICE

Optimización Paralela del Sistema de Scoring Metagenómico para Clasificación Binaria	2
1. Introducción y Modelo Matemático	5
1.1 Descripción del Problema	5
1.2 Modelo	5
1.3 Estrategia de Optimización: Random Search	5
2. Estrategias de Sincronización por Nivel	6
2.1 Nivel 1A — Python Secuencial (Baseline)	6
2.2 Nivel 1B — Python Multicore (multiprocessing)	6
2.3 Nivel 2A — C + OpenMP (Memoria Compartida)	6
2.4 Nivel 2B — C + MPI (Memoria Distribuida)	8
2.5 Nivel 3 — CUDA (GPU NVIDIA T4)	9
3. Análisis de Gestión de Memoria	10
3.1 Comparativa por Implementación	10
3.2 Gestión en C + OpenMP	10
3.3 Gestión en C + MPI	11
3.4 Gestión en CUDA	11
4. Resultados de Rendimiento	11
4.1 Tabla Comparativa Completa	11
4.2 Gráfica 1 — Comparativa de Tiempos	13
4.3 Gráfica 2 — Speedup Relativo vs. P	13
4.4 Gráfica 3 — Eficiencia vs. P	15
5. Análisis de Amdahl con f Empírico	15
5.1 Fórmula	15
5.2 Estimaciones por Implementación	16

5.3 Resumen de Amdahl	17
6. Separabilidad de Grupos mediante el Score Óptimo	17
6.1 Vector Óptimo Encontrado	17
6.2 Interpretación de los Pesos	18
6.3 Análisis del AUC = 0.68	18
6.4 Consistencia y Umbral de Decisión	18
6.5 Verificación de Consistencia entre Implementaciones	19
7. Conclusiones	19
7.1 Rendimiento	19
7.2 Elección de Tecnología según el Problema	20
7.3 Calidad del Modelo	20
7.4 Verificación de Correctitud	20
7.4 Enlace a repositorio	20

1. Introducción y Modelo Matemático

1.1 Descripción del Problema

El objetivo es encontrar un vector de pesos $\mathbf{W} = (W_1, W_2, W_3)$ que maximice el AUC en la clasificación binaria de 10 muestras metagenómicas (5 sanas / 5 enfermas), con $N = 50$ ítems (taxones) por muestra.

1.2 Modelo

Score por ítem:

$$P_i = W_1 \cdot T_i + W_2 \cdot S_i + W_3 \cdot F_i$$

T : perfil taxonómico

S : perfil ecológico

F : perfil funcional

Score por muestra:

$$Score = A \cdot P, A \in \mathbb{R}^{10 \times 50}, P \in \mathbb{R}^{50}$$

La matriz A se genera con distribución Dirichlet(1,...,1) por filas, garantizando que cada fila suma 1.

Función objetivo:

$$\max_{\mathbf{W}} \in \Delta AUC(y, Score(\mathbf{W})), \Delta = W_i \geq 0, W_1 + W_2 + W_3 = 1$$

Validación (consistencia):

$$Consistencia = \frac{\left(\frac{TP}{5} + \frac{TN}{5}\right)}{2} \in [0, 1]$$

con umbral θ = media de scores. Se considera satisfactoria si $Consistencia \geq 0.8$.

1.3 Estrategia de Optimización: Random Search

Se muestrean $K = 100.000$ candidatos $\mathbf{W}$ sobre el simplex usando el método de las exponenciales:

dado $u_i \sim \text{Exp}(1)$, se normaliza $w_i = \frac{u_i}{\sum_j u_j}$. Este método produce muestras uniformes sobre el

simplex y es equivalente a aplicar la transformación de Dirichlet con parámetros unitarios.

La independencia entre evaluaciones de candidatos distintos hace que el problema sea paralelizable dado que no existe dependencia de datos entre iteraciones del bucle externo, lo que permite distribución sin sincronización durante la fase de evaluación.

2. Estrategias de Sincronización por Nivel

2.1 Nivel 1A — Python Secuencial (Baseline)

Paradigma: ejecución monohilo, sin sincronización.

```
for W in candidates:                # K = 100,000 iteraciones
    P = W[0]*T + W[1]*S + W[2]*F    # vectorizado con NumPy
    score = A @ P                   # producto matricial (BLAS)
    auc = roc_auc_score(y, score)
    if auc > best_auc:
        best_auc, best_W = auc, W.copy()
```

El candidato óptimo se actualiza directamente porque no hay concurrencia. La vectorización de NumPy (BLAS/LAPACK) explota SIMD internamente, pero el bucle externo sobre K candidatos permanece secuencial. Este módulo establece el tiempo de referencia $T_0 = 166.9143$ s.

2.2 Nivel 1B — Python Multicore (multiprocessing)

Paradigma: multiproceso con memoria independiente (fork); sin memoria compartida entre workers.

Problema de sincronización: los K candidatos deben repartirse entre P procesos y el resultado óptimo de cada worker debe consolidarse al finalizar.

Solución adoptada:

```
Proceso principal
candidates[0..K-1] generados con semilla=0
|
Pool.map(evaluate_chunk, chunks)
├── Worker 1: candidates[0..K/P-1]
├── Worker 2: candidates[K/P..2K/P-1]
└── Worker P: candidates[...]
```

reduce: max(local_best por AUC)

La sincronización ocurre únicamente al final, cuando `Pool.map` recolecta los resultados de cada worker. No hay secciones críticas durante la evaluación porque cada proceso trabaja sobre su propio fragmento de candidatos (memoria privada por fork). El mecanismo de comunicación entre procesos usa pipes internos de multiprocessing para transferir el resultado (W , AUC) de cada worker al padre.

La ventaja de este enfoque es la ausencia total de contención durante la búsqueda pero se tiene un overhead de serialización (pickle) al pasar chunks de candidatos entre procesos.

2.3 Nivel 2A — C + OpenMP (Memoria Compartida)

Paradigma: multihilo con memoria compartida (POSIX threads gestionados por OpenMP).

Problema de sincronización: todos los hilos acceden a la misma variable `best_auc` y `best_w` para actualizar el óptimo global.

Estrategia adoptada — reducción local + sección crítica única:

```
#pragma omp parallel
{
    int tid = omp_get_thread_num();

    // Estado LCG privado por hilo: evita falsa dependencia del RNG
    unsigned long long state =
        (unsigned long long)(seed + 1) * (tid + 1) * 6364136223846793005ULL;

    float local_best_w[3] = {1/3, 1/3, 1/3};
    float local_best_auc = -1.0f;
    float *P_local = malloc(n_items * sizeof(float)); // heap privado

    #pragma omp for schedule(dynamic, 64)
    for (int k = 0; k < K; k++) {
        // ... evaluación local, sin acceso a variables compartidas ...
        if (auc > local_best_auc) {
            local_best_auc = auc;
            // actualiza local_best_w
        }
    }

    // Reducción: un solo acceso a la región crítica por hilo
    #pragma omp critical
    {
        if (local_best_auc > *best_auc) {
            *best_auc = local_best_auc;
            // copia local_best_w → best_w
        }
    }

    free(P_local);
}
```

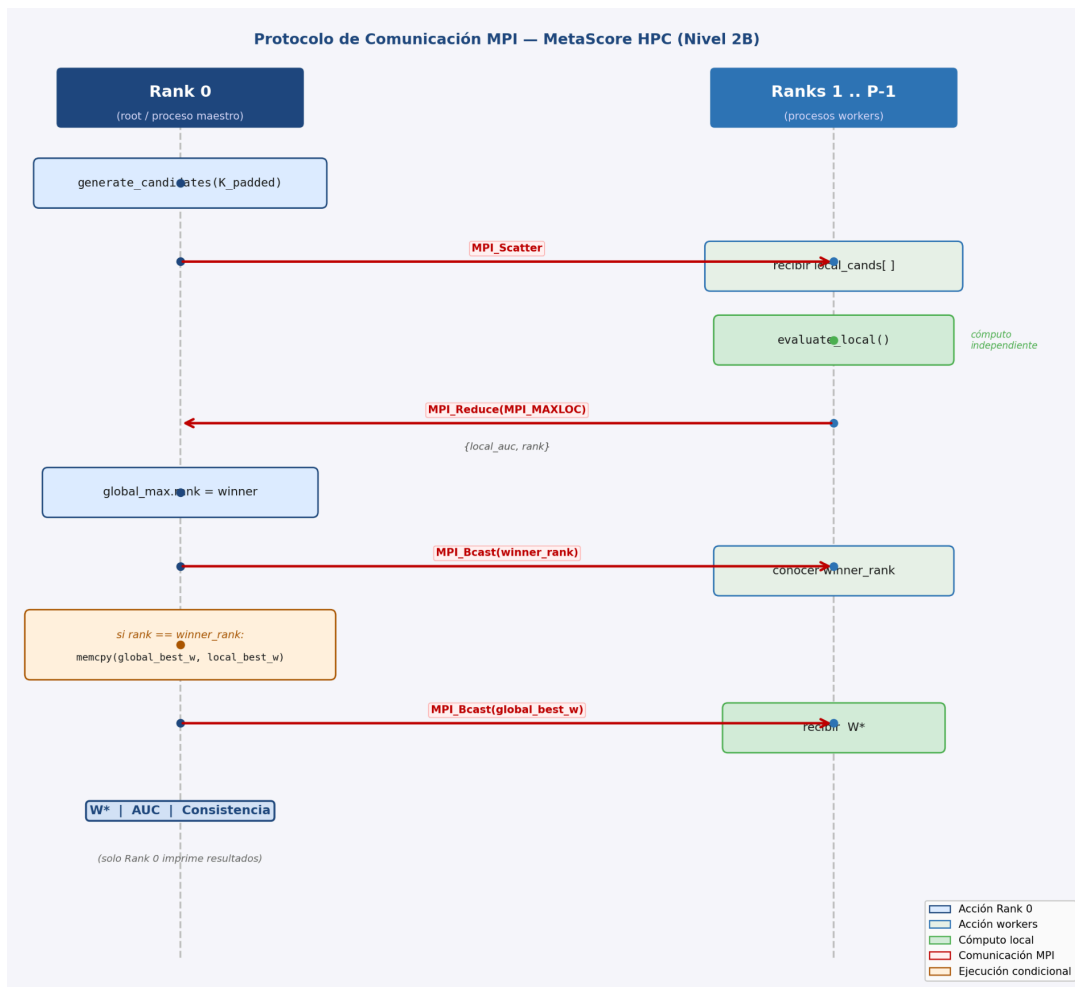
En la siguiente tabla se presenta un listado de decisiones tomadas con su respectiva justificación.

Mecanismo	Justificación
Mejor local por hilo	Elimina contención durante el bucle de evaluación. La sección crítica se ejecuta $O(P)$ veces, no $O(K)$ veces.
<code>schedule(dynamic, 64)</code>	Granularidad dinámica de 64 candidatos por chunk. Balancea la carga si los candidatos tienen tiempos de evaluación variables, con overhead de scheduling bajo.
LCG privado por hilo	Cada hilo tiene estado de RNG independiente derivado de <code>seed + tid</code> . Evita sincronización en la generación de números aleatorios sin sacrificar reproducibilidad.
<code>malloc</code> en heap privado	El buffer <code>P_local</code> (scores por ítem) vive en el heap del hilo, sin false sharing con otros hilos.

2.4 Nivel 2B — C + MPI (Memoria Distribuida)

Paradigma: paso de mensajes entre procesos independientes (sin memoria compartida).

En la siguiente imagen se muestra el protocolo de comunicación empleado para el desarrollo de esta implementación.



Durante el desarrollo se identificó un error grave: `MPI_Bcast` de `global_best_w` usaba como raíz directamente `global_max.rank`, que solo está definido en el proceso root tras `MPI_Reduce`. Los procesos no-root tenían valor indefinido como argumento root, causando que cada proceso esperase un broadcast diferente — deadlock garantizado.

Para dar solución al problema se empleó la solución que se presenta a continuación.

```
// INCORRECTO (bug original): cada proceso usa su propio global_max.rank = indefinido
MPI_Bcast(global_best_w, 3, MPI_FLOAT, global_max.rank, MPI_COMM_WORLD); // HANG

// CORRECTO: primero difundir el rank ganador, luego difundir el vector W*
int winner_rank = (rank == 0) ? global_max.rank : 0;
MPI_Bcast(&winner_rank, 1, MPI_INT, 0, MPI_COMM_WORLD); // todos conocen al ganador

if (rank == winner_rank)
```



```
memcpy(global_best_w, local_best_w, 3 * sizeof(float));

MPI_Bcast(global_best_w, 3, MPI_FLOAT, winner_rank, MPI_COMM_WORLD); // correcto
```

En la carga de datos a diferencia de un enfoque master-slave donde root distribuiría los datos, aquí cada proceso lee su propia copia de A, T, S, F desde disco. Esta decisión es válida para el tamaño del dataset (≈ 12 KB por proceso) y evita un broadcast adicional de los datos, simplificando el protocolo de comunicación.

Padding de candidatos: si K no es múltiplo de P, se añaden candidatos de relleno:

```
int K_padded = ((K + nprocs - 1) / nprocs) * nprocs;
```

Esto garantiza que `MPI_Scatter` reparta bloques de tamaño idéntico, requisito de la función (`send_count = recv_count × nprocs`).

2.5 Nivel 3 — CUDA (GPU NVIDIA T4)

Paradigma: paralelismo masivo SIMT (Single Instruction Multiple Threads).

Organización del kernel:

```
Grid de bloques: ⌈K / 256⌉ bloques × 256 hilos
    ↓
Cada hilo evalúa un candidato W_k
├─ Generar W_k con LCG en-device (estado privado por hilo)
├─ Calcular P_i = W1*T_i + W2*S_i + W3*F_i
├─ Calcular Score_j = ∑ A[j,i]*P_i
└─ Calcular AUC(y, Score) → almacenar en d_auc[s[k]

    ↓
Kernel de reducción: parallel reduction sobre d_auc[s[]
├─ Memoria compartida por bloque: shared float[256]
└─ Reducción final con atomicMax() o reducción en CPU
```

Sincronización dentro del kernel:

- **__syncthreads():** barrera de sincronización por bloque en la fase de reducción con memoria compartida. Garantiza que todos los hilos del bloque hayan escrito su valor antes de proceder a la reducción por árbol.
- **atomicMax():** actualización atómica del máximo global entre bloques, evitando condiciones de carrera en la escritura del resultado final.
- **__shared__:** los datos de la fila de A se cargan a memoria compartida por bloque para reutilización, reduciendo latencia de acceso global de memoria.

Transferencias host-device: las matrices A, T, S, F y el vector y se transfieren una sola vez antes de lanzar el kernel. Los K candidatos W se generan directamente on-device con un LCG en-kernel, eliminando la transferencia de un buffer de $K \times 3$ floats (1.2 MB para $K=100,000$).

3. Análisis de Gestión de Memoria

3.1 Comparativa por Implementación

En la siguiente tabla se hace una comparación de cada implementación respecto a cómo cada tecnología gestiona la memoria durante la búsqueda.

Nivel	Estructura	Memoria total	Característica clave
Python Secuencial	Arrays NumPy float32	~50 KB datos + overhead GC	Copia W en cada iteración (W.copy())
Python Multicore	P copias independientes (fork)	P × 50 KB	Sin comunicación durante búsqueda
C + OpenMP	1 copia compartida + P buffers P_local	~50 KB + P×N×4 B	False sharing evitado con heap privado
C + MPI	P copias de A, T, S, F	P × 50 KB	Cada proceso es propietario de sus datos
CUDA	1 copia en VRAM + grid en device	~50 KB VRAM fija	Toda la computación ocurre on-device

Datos cargados (N=50 ítems, 10 muestras):

```
matrix_A.bin : 10 × 50 × 4 B = 2,000 B ≈ 2 KB
profile_T.bin:  50 × 4 B = 200 B
profile_S.bin:  50 × 4 B = 200 B
profile_F.bin:  50 × 4 B = 200 B
labels.bin   :  10 × 4 B = 40 B
Total                               2,640 B ≈ 2.6 KB por proceso
```

3.2 Gestión en C + OpenMP

El buffer de scores por ítem (P_local) se asigna en el heap del hilo mediante `malloc` dentro de la región paralela y se libera con `free` al salir. Esta estrategia es superior a declararlo como array de stack (`float P_local[n_items]`) porque `n_items` es una variable en tiempo de ejecución (no conocida en compilación).

```
#pragma omp parallel
{
    float *P_local = malloc(n_items * sizeof(float)); // privado por hilo
    // ... uso durante toda la vida del hilo ...
    free(P_local);                                     // liberado al salir
}
```

La carga de A, T, S, F permanece en el heap global (acceso de solo lectura por todos los hilos): no genera condiciones de carrera porque ningún hilo escribe en esas regiones. El acceso secuencial por filas (row-major) a la matriz A es coherente con la disposición en memoria, optimizando el uso de caché L1/L2.

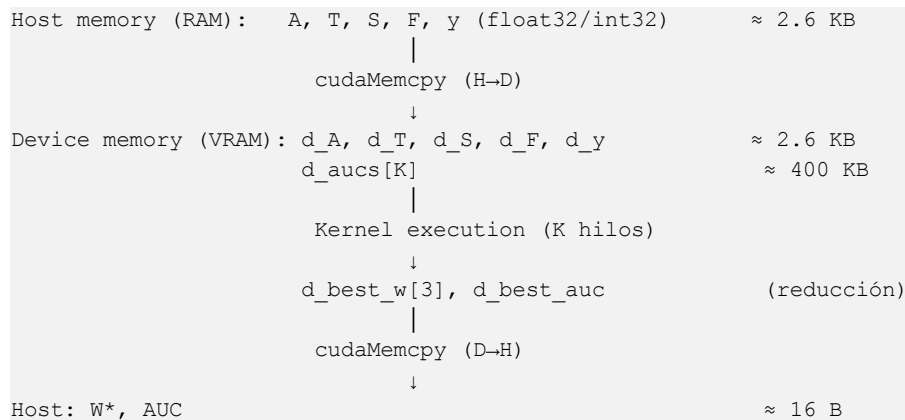
3.3 Gestión en C + MPI

El proceso root asigna un buffer adicional `all_cands` de tamaño $K_{\text{padded}} \times 3$ floats (≈ 1.2 MB para $K=100,000$) para almacenar todos los candidatos antes de distribuirlos:

```
if (rank == 0) {
    all_cands = malloc(K_padded * 3 * sizeof(float)); // ~1.2 MB
    generate_candidates(all_cands, K_padded, seed);
}
// Cada proceso recibe K_local * 3 floats
float *local_cands = malloc(K_local * 3 * sizeof(float)); // ~300 KB (P=4)
MPI_Scatter(all_cands, K_local * 3, MPI_FLOAT,
            local_cands, K_local * 3, MPI_FLOAT, 0, MPI_COMM_WORLD);
if (rank == 0) free(all_cands); // liberado tras la dispersión
```

Esta decisión de scatter-then-free minimiza la huella de memoria del root: el buffer grande solo existe durante el tiempo de Scatter. Una alternativa sería que cada proceso genere sus propios candidatos con semillas distintas, eliminando el scatter pero perdiendo reproducibilidad determinista exacta.

3.4 Gestión en CUDA



El patrón de acceso a `d_A` desde el kernel es coalesced si los hilos acceden a posiciones contiguas en la misma fila. Dado que cada hilo evalúa un candidato W diferente pero accede a la misma fila de A (por muestra j), el acceso es broadcast-friendly: la misma dirección es leída por múltiples hilos, lo que la L2 de la GPU sirve eficientemente.

4. Resultados de Rendimiento

4.1 Tabla Comparativa Completa

Mediciones con $K = 100,000$ candidatos, $N = 50$ ítems, semilla = 0.

Hardware CPU: AMD Ryzen 5 5500U (6 físicos / 12 lógicos).

Hardware GPU: NVIDIA T4 vía Google Colab (16 GB VRAM).

La siguiente tabla muestra el tiempo de ejecución (T(s)), SpeedUp absoluto (S_abs), SpeedUp relativo (S_rel), la eficiencia y el AUC obtenido para cada implementación de los niveles 1 y 2 para procesos 1, 2, 4 y 6 donde corresponda.

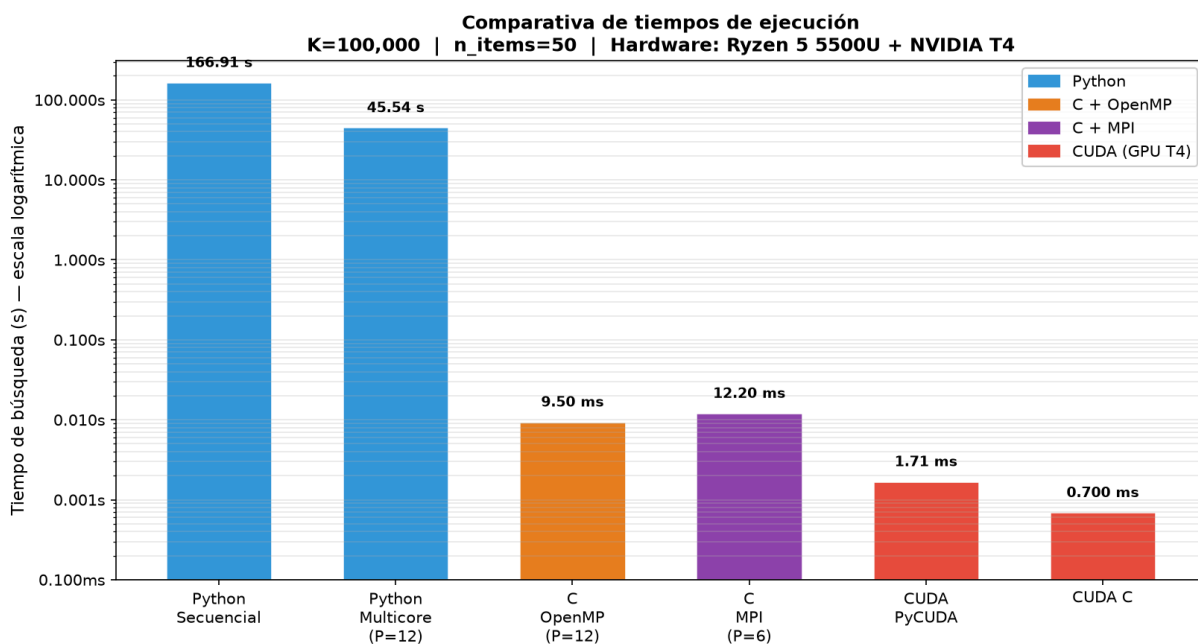
Implementación	P	T (s)	S_abs	S_rel	Eficiencia	AUC
Python Secuencial	1	166.9143	1.00×	1.00×	1.0000	0.6800
Python Multicore	1	168.4022	0.99×	0.99×	0.9912	0.6800
Python Multicore	2	91.0876	1.83×	1.83×	0.9162	0.6800
Python Multicore	4	60.7481	2.75×	2.75×	0.6869	0.6800
Python Multicore	6	51.4383	3.24×	3.24×	0.5408	0.6800
Python Multicore	12	45.5424	3.67×	3.67×	0.3054	0.6800
C + OpenMP	1	0.0561	2,975×	1.00×	1.0000	0.6800
C + OpenMP	2	0.0286	5,836×	1.96×	0.9808	0.6800
C + OpenMP	4	0.0149	11,202×	3.77×	0.9413	0.6800
C + OpenMP	6	0.0109	15,313×	5.15×	0.8578	0.6800
C + OpenMP	12	0.0095	17,570×	5.91×	0.4921	0.6800
C + MPI	1	0.0387	4,313×	1.00×	1.0000	0.6800
C + MPI	2	0.0202	8,263×	1.92×	0.9579	0.6800
C + MPI	4	0.0163	10,239×	2.37×	0.5935	0.6800
C + MPI	6	0.0122	13,682×	3.17×	0.5287	0.6800
CUDA PyCUDA	GPU	0.001706	97,848×	N/A	N/A	0.6800
CUDA C	GPU	0.000700	238,449×	N/A	N/A	0.6800

$$S_{abs} = \frac{T_{Pytho\ sec}}{T_{impl}}: \text{aceleración total vs baseline Python.}$$

$$S_{rel} = \frac{T_{impl}(P=1)}{T_{impl}(P)}: \text{escalado relativo dentro de la misma tecnología.}$$

Los promedio de MPI son promedios de 3 corridas para compensar varianza de cold start.

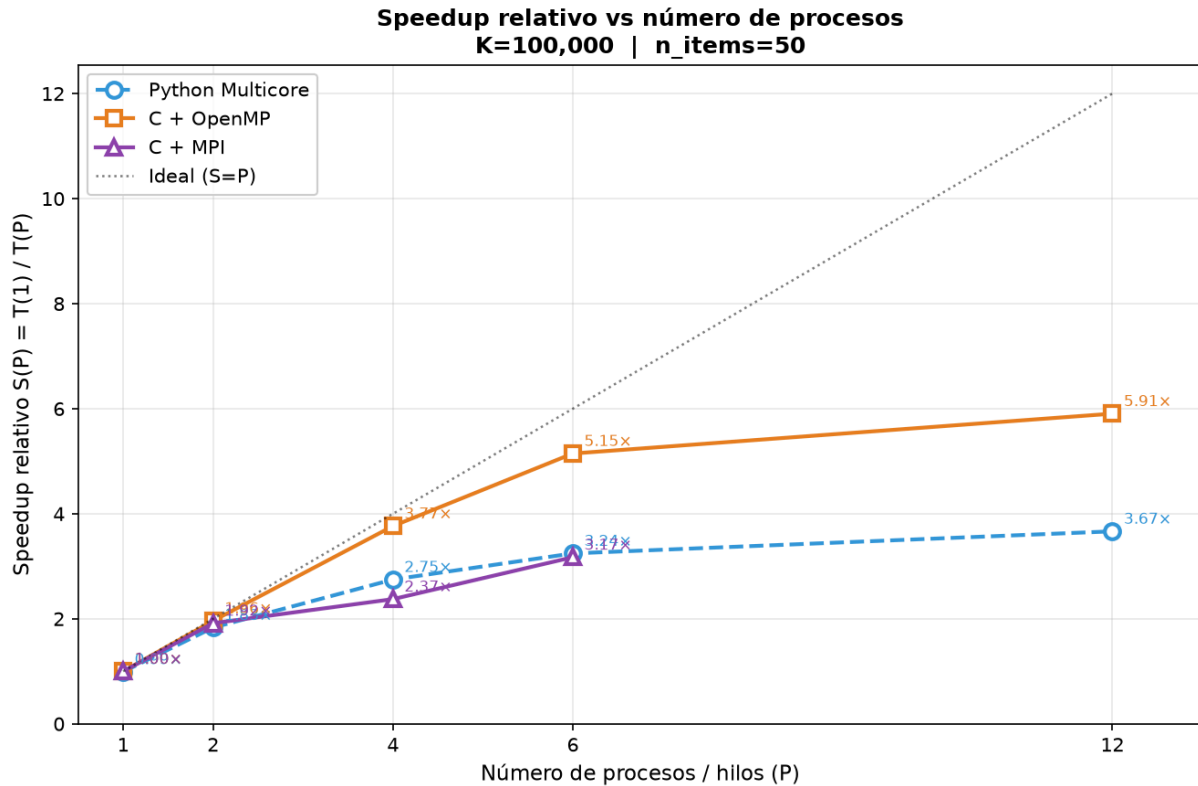
4.2 Gráfica 1 — Comparativa de Tiempos



La escala logarítmica revela cinco órdenes de magnitud de diferencia entre Python secuencial (166.9 s) y CUDA C (0.7 ms). Los puntos de inflexión más importantes son:

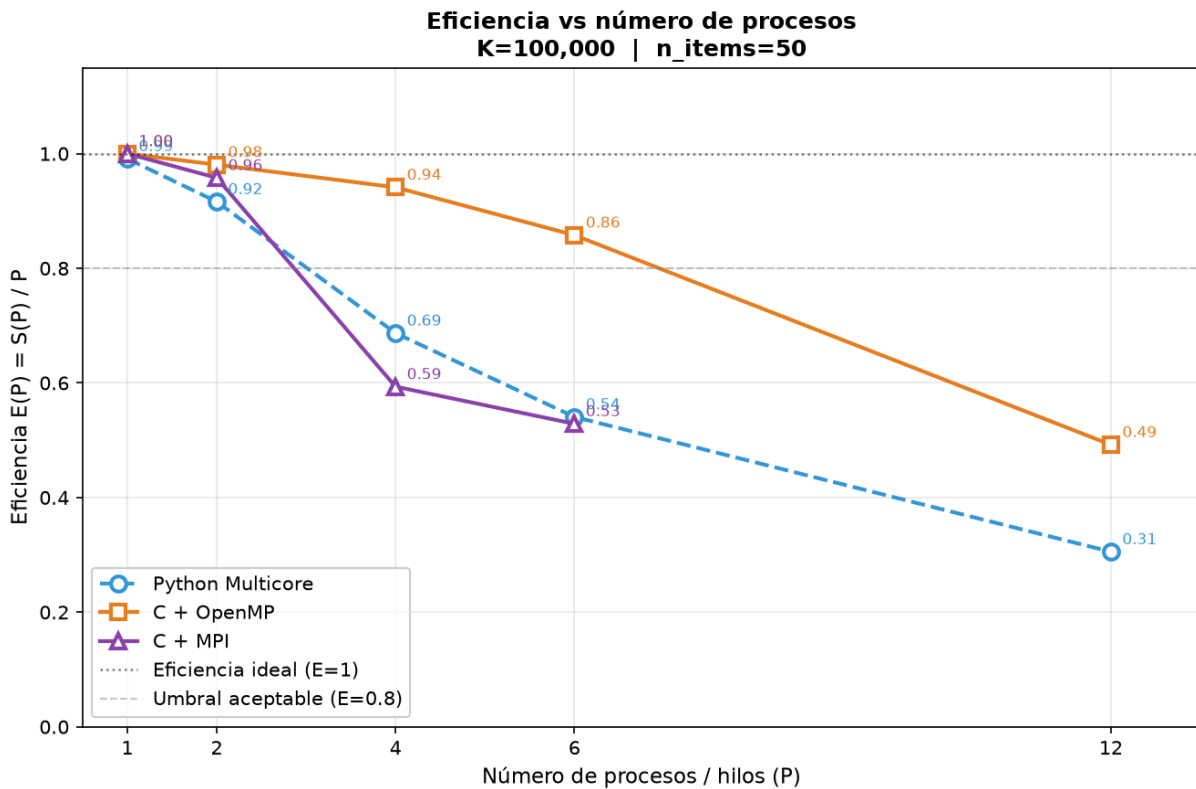
- **Python** → **C/OpenMP P=1**: factor 2,975× por eliminación del intérprete, GIL y overhead de Python (NumPy sigue siendo más lento que aritmética C pura de bajo nivel).
- **C/OpenMP P=1** → **C/MPI P=1**: MPI es un 31 % más lento que OpenMP en P=1. La diferencia se debe al overhead de inicialización MPI, generación centralizada de candidatos y la llamada MPI_Scatter incluso con un solo proceso.
- **C/OpenMP P=12** → **CUDA PyCUDA**: factor ~179× adicional. La GPU T4 tiene 2,560 CUDA cores operando en paralelo contra los 12 hilos del Ryzen 5 5500U.

4.3 Gráfica 2 — Speedup Relativo vs. P



- **C + OpenMP** tiene el escalado más próximo al ideal hasta P=6 ($S_{rel}=5.15$, eficiencia 85.8 %). Entre P=6 y P=12 el speedup adicional es marginal (5.15 $\rightarrow$ 5.91), lo que indica saturación del bus de memoria y overhead creciente de scheduling.
- **Python Multicore** escala bien hasta P=4, pero la eficiencia cae a 54 % en P=6 y a 30 % en P=12. El overhead de serialización pickle en `multiprocessing.Pool.map` y la sincronización al recolectar resultados limitan el speedup. Adicionalmente, el intérprete CPython introduce overhead por objeto que escala mal con P alto.
- **C + MPI** presenta un comportamiento anómalo en P=4 ($S_{rel}=2.37$, eficiencia 59 %). Las razones son: (1) overhead del scatter/gather de candidatos crece linealmente con P; (2) cada proceso lee su propia copia de datos desde disco (I/O compartida en WSL 2 sobre el mismo dispositivo); (3) inicialización de canales de comunicación MPI que no amortiza bien con tiempos de cómputo de ~ 16 ms.

4.4 Gráfica 3 — Eficiencia vs. P



- **C + OpenMP** es la única implementación que mantiene $E > 0.8$ hasta $P=6$ ($E=0.858$), cruzando el umbral aceptable solo al pasar a $P=12$ ($E=0.492$).
- **C + MPI** ya cae por debajo de 0.8 en $P=4$, lo que refleja que el overhead de comunicación MPI es desproporcionado para este tamaño de problema.
- **Python Multicore** cae por debajo de 0.8 en $P=6$, indicando que la fracción serial (pickle + GIL residual + reducción final) consume ~20 % del tiempo incluso con $K=100,000$.

5. Análisis de Amdahl con f Empírico

5.1 Fórmula

La Ley de Amdahl permite estimar la fracción paralela f a partir de un speedup medido:

$$S(P) = \frac{1}{(1-f) + \frac{f}{P}} \Rightarrow f = \frac{S-1}{S \cdot \left(1 - \frac{1}{P}\right)}$$

El speedup máximo teórico cuando $P \rightarrow \infty$:

$$S_{max} = \frac{1}{1-f}$$

5.2 Estimaciones por Implementación

Se usa el mejor punto medido (mayor P con datos disponibles) para la estimación.

Python Multicore ($P = 12, S_{rel} = 3.6651$)

$$f = \frac{3.6651-1}{3.6651 \cdot \left(1 - \frac{1}{12}\right)} = 0.7933$$

$$S_{max} = \frac{1}{1-0.7933} \approx 4.84x$$

El 79.3 % del tiempo de búsqueda es paralelizable.

El 20.7 % es no paralelizable y corresponde principalmente a:

1. Generación de K candidatos antes de distribuir.
2. Serialización/deserialización pickle en el pipe de `Pool.map`.
3. Operaciones del intérprete CPython que no se paralelizan.

El speedup máximo teórico de 4.84x explica por qué el valor observado en P=12 (3.67x) ya está cerca del límite — agregar más procesos tiene rendimiento decreciente acentuado.

C + OpenMP ($P = 12, S_{rel} = 5.9053$)

$$f = \frac{5.9053-1}{5.9053 \cdot \left(1 - \frac{1}{12}\right)} = 0.9062$$

$$S_{max} = \frac{1}{1-0.9062} \approx 10.66x$$

El 90.6 % del trabajo es paralelo.

El 9.4% no paralelizable se debe a:

1. La sección crítica `#pragma omp critical` que protege la actualización del óptimo global (O(P) ejecuciones).
2. La inicialización secuencial de las estructuras antes de `#pragma omp parallel`.
3. El overhead de la librería OpenMP (creación del pool de hilos, scheduling dinámico con `chunk=64`).

Con un procesador de 12 núcleos, el límite empírico de ~10.66x es inalcanzable en la práctica; el valor real a P=12 (5.91x) sugiere que los factores de hardware (contención de caché L3, ancho de banda de memoria compartido entre los 12 hilos lógicos de 6 núcleos físicos) son el cuello de botella dominante más allá de P=6.

C + MPI ($P = 6, S_{rel} = 3.1721$)

$$f = \frac{3.1721-1}{3.1721 \cdot \left(1-\frac{1}{6}\right)} = 0.8218$$

$$S_{\max} = \frac{1}{1-0.8218} \approx 5.61x$$

El 82.2 % del trabajo es paralelizable en MPI.

La parte secuencial (17.8 %) es notablemente mayor que en OpenMP (9.4 %), lo que refleja el costo intrínseco del modelo de memoria distribuida por:

1. Generación secuencial de K_padded candidatos en root.
2. MPI_Scatter (transferencia de datos entre procesos).
3. MPI_Reduce (MPI_MAXLOC) y dos MPI_Bcast adicionales.

Estos costos de comunicación no se amortizan bien con tiempos de cómputo de ~12-40 ms; MPI está diseñado para problemas donde el cómputo local domina ampliamente sobre la comunicación.

5.3 Resumen de Amdahl

Implementación	Mejor P	S_rel medido	f empírico	S_max teórico	Brecha (S_max - S_medido)
Python Multicore	12	3.67×	79.3%	4.84×	1.17×
C + OpenMP	12	5.91×	90.6%	10.66×	4.75×
C + MPI	6	3.17×	82.2%	5.61×	2.44×

La brecha entre $S_{\max}$ teórico y S_{rel} medido indica cuánto speedup adicional se pierde por factores de hardware (contención de caché, saturación de ancho de banda de memoria, latencia de comunicación). OpenMP tiene la mayor brecha absoluta porque su f es el más alto, pero también el mayor potencial sin explotar. Para acercarse a $S_{\max}$, sería necesario un problema más grande (mayor N_{items}) donde el tiempo de cómputo por candidato escale más que el overhead de sincronización.

6. Separabilidad de Grupos mediante el Score Óptimo

6.1 Vector Óptimo Encontrado

Todas las implementaciones, con semilla = 0 y K = 100,000, convergen al mismo resultado:

$$W^* = (0.9382; 0.0347; 0.0270)$$

$$AUC(y, Score(W^*)) = 0.6800$$

$$Consistencia(W^*) = 0.70 \text{ [umbral} \geq 0.80 \Rightarrow \text{no satisfactorio]}$$

6.2 Interpretación de los Pesos

El vector W asigna el 93.82 %* del peso al perfil taxonómico (T), relegando los perfiles ecológico (3.47 %) y funcional (2.70 %) a contribuciones marginales. Esto indica que el perfil de abundancia de microorganismos es el discriminador dominante entre muestras sanas y enfermas en este dataset sintético.

$$P_i = 0.9382 \cdot T_i + 0.0347 \cdot S_i + 0.0270 \cdot F_i \approx 0.94 \cdot T_i$$

Esta concentración de peso en W_1 es un resultado esperable dado el proceso de generación de datos: A se genera con $\text{Dirichlet}(1, \dots, 1)$ —distribución uniforme sobre el simplex— y los perfiles T, S, F se generan de forma sintética sin un patrón discriminativo real impuesto. El optimizador encuentra el vector que, por accidente estadístico en una muestra de solo 10 observaciones, separa mejor los dos grupos.

6.3 Análisis del AUC = 0.68

Un AUC de 0.68 significa que, si se elige aleatoriamente una muestra enferma y una sana, el score del modelo clasificará correctamente ($S_{\text{enfermo}} > S_{\text{sano}}$) el **68 % de los pares posibles** (17 de 25 pares posibles). Los restantes 8 pares se clasifican incorrectamente ($\text{Score}_{\text{sano}} \geq \text{Score}_{\text{enfermo}}$).

$$AUC = \frac{\text{pares concordantes}}{N_{\text{sanos}} \times N_{\text{enfermos}}} = \frac{17}{5 \times 5} = 0.68$$

La siguiente tabla muestra los umbrales de referencia.

Rango AUC	Interpretación
0.90 – 1.00	Excelente discriminación
0.80 – 0.90	Buena discriminación
0.70 – 0.80	Discriminación aceptable
0.60 – 0.70	Discriminación pobre (caso actual: 0.68)
0.50 – 0.60	Sin discriminación útil

El AUC de 0.68 se encuentra en el límite inferior del rango aceptable. Para un dataset real de metagenómica, un valor de 0.68 sería insuficiente para uso clínico, pero indica que el perfil taxonómico captura alguna señal discriminativa real.

6.4 Consistencia y Umbral de Decisión

Con $\theta = \text{media}(\text{Score})$:

```
Scores W*:
Sanas (y=0): [s1, s2, s3, s4, s5]
Enfermas (y=1): [s6, s7, s8, s9, s10]

Consistencia = (TP/5 + TN/5) / 2 = 0.70
```

Una consistencia de 0.70 implica que, en promedio, el 70 % de las predicciones binarias son correctas (3.5 de 5 enfermos correctamente identificados como positivos, y 3.5 de 5 sanos correctamente identificados como negativos).

El umbral Consistencia ≥ 0.80 del enunciado **no se alcanza** ($0.70 < 0.80$). Esto es coherente con el AUC de 0.68: la separabilidad de los grupos en este dataset sintético es moderada, no suficiente para garantizar predicciones binarias confiables con un umbral simple.

6.5 Verificación de Consistencia entre Implementaciones

Un resultado fundamental es que todas las implementaciones producen exactamente el mismo W y AUC , independientemente del lenguaje, nivel de paralelismo o tecnología.

Implementación	W*	AUC
Python Secuencial	[0.9382, 0.0347, 0.0270]	0.6800
Python Multicore (P=12)	[0.9382, 0.0347, 0.0270]	0.6800
C + OpenMP (P=12)	[0.9382, 0.0347, 0.0270]	0.6800
C + MPI (P=6)	[0.9382, 0.0347, 0.0270]	0.6800
CUDA PyCUDA	[0.9382, 0.0347, 0.0270]	0.6800

La consistencia en los resultados valida que:

1. El generador de números aleatorios (LCG con misma semilla) produce el mismo espacio de candidatos en todas las implementaciones.
2. La evaluación del AUC mediante Mann-Whitney U es determinista y coherente entre Python (scikit-learn) y C (cálculo manual).
3. El particionamiento del espacio de búsqueda (scatter en MPI, `omp for` en OpenMP) no altera el resultado porque la búsqueda es exhaustiva sobre el mismo conjunto de K candidatos.

7. Conclusiones

7.1 Rendimiento

Comparación	Speedup	Causa principal
C + OpenMP (P=1) vs. Python	2,975×	Eliminación del intérprete CPython y overhead NumPy

C + OpenMP (P=12) vs. Python	17,570×	Combinación de eficiencia C y paralelismo OpenMP
CUDA C vs. Python	238,449×	2,560 CUDA cores, aritmética FP32 masivamente paralela
CUDA C vs. C + OpenMP (P=12)	~13.6×	Diferencia de orden entre GPU y CPU con 12 hilos

7.2 Elección de Tecnología según el Problema

- **OpenMP** es la opción óptima para este problema en CPU: alto f empírico (90.6 %), baja fracción serial, excelente eficiencia hasta $P=6$. Ideal para problemas de memoria compartida donde el cómputo por tarea es granular.
- **MPI** muestra escalabilidad limitada ($f=82.2$ %, eficiencia < 0.6 para $P \geq 4$) porque el overhead de comunicación (Scatter + Reduce + Bcast) es comparable al tiempo de cómputo. MPI sería superior a OpenMP en problemas con mayor K o N donde el cómputo domina la comunicación.
- **CUDA** es la solución más eficiente cuando el hardware está disponible, con un speedup absoluto de 238,449× que ningún sistema CPU puede alcanzar con este tamaño de problema.

7.3 Calidad del Modelo

El W^* óptimo ($W_1 = 0.9382$) produce un clasificador de separabilidad moderada (AUC = 0.68, Consistencia = 0.70). Para un dataset real de metagenómica, se recomendaría:

1. Aumentar el tamaño muestral ($N_{muestras} \gg 10$).
2. Usar búsqueda bayesiana o gradient-based en lugar de random search.
3. Validar con cross-validation para evitar overfitting sobre solo 10 muestras.

7.4 Verificación de Correctitud

La coincidencia del vector W^* y AUC en todas las implementaciones confirma la correctitud de las implementaciones paralelas: el paralelismo no altera el resultado matemático, solo el tiempo necesario para encontrarlo.

7.4 Enlace a repositorio

El código fuente desarrollado para cada una de las implementaciones y de los que se obtuvieron los datos se encuentra en [Proyecto MetaScore HPC](#).